

Projet d'Ingénierie de 2^{ème} année
Arts et Métiers — Campus d'Aix-en-Provence
Automne 2025 – Printemps 2026

ScieWay — Conception et réalisation d'un mini-véhicule autonome 1/10^{ème}

AWAKE Challenge 2026



Figure 1 : Prototype final ScieWay équipé du LiDAR LD19, des quatre HC-SR04 et de l'électronique embarquée.

Présenté par :
AZZAL Haitam
BAHIR El Mahdi
AKBIB BUKRABA Youssef

Encadré par : M. GOMAND Julien — M. FAVAREL Camille

Résumé

Ce rapport présente la conception, la modélisation, la commande et la réalisation d'un véhicule autonome à l'échelle 1/10^{ème} pour l'AWAKE Challenge 2026. L'architecture distribuée associe un Raspberry Pi 5 (perception et décision Follow The Gap) à un ESP32 (asservissement temps réel PWM, sécurité ultrasons). Après identification expérimentale de la chaîne de propulsion, modélisation cinématique d'Ackermann et fusion Kalman IMU/Hall, le véhicule a été validé en autonomie complète à 1,5 m/s sur circuit d'essai, sans intervention sur cinq tours. Les tentatives de localisation par scan matching (ICP) ayant échoué par dérive cumulée, la stratégie finale repose sur une navigation réactive FTG pure, robuste et entièrement opérationnelle.

Sommaire

Sommaire	1
1. Introduction et contexte	3
1.1 Le défi AWAKE Challenge 2026	3
1.2 Objectifs du projet.....	3
1.3 Périmètre et démarche.....	3
1.4 Présentation de l'équipe et répartition des rôles	4
2. Architecture du véhicule	5
2.1 Vue d'ensemble du système.....	5
2.2 Caractéristiques mécaniques et électriques.....	5
2.3 Chaîne de perception et de calcul	6
2.4 Pile logicielle	6
3. Capteurs et perception.....	7
3.1 LiDAR LD19 — perception principale	7
3.2 Capteurs ultrasons HC-SR04 — sécurité de proximité	8
3.3 Centrale inertielle MPU-6050.....	8
3.4 Capteur à effet Hall — odométrie de précision	8
4. Identification et modélisation	9
4.1 Identification expérimentale de la chaîne de propulsion	9
4.2 Modélisation cinématique : modèle bicyclette d'Ackermann	10
4.3 Modèle dynamique et calcul de l'angle de commande.....	11
4.4 Fusion de capteurs par filtre de Kalman	12
5. Synthèse de la commande : Follow The Gap et PID	14
5.1 Pourquoi Follow The Gap ?.....	14
5.2 L'algorithme Follow The Gap pas à pas.....	14
5.3 Asservissement bas niveau : PID vitesse et direction.....	15
5.3.1 PID de vitesse	15
5.3.2 PID de direction	16
5.4 Réglage des PID sous MATLAB.....	16
5.5 Sécurité multi-capteurs	16
Couche 1 — LiDAR (principale).....	16
Couche 2 — Ultrasons (redondance).....	16
Couche 3 — Limites logicielles.....	16

6. Cartographie LiDAR et tentative de localisation.....	17
6.1 Visualisation sous ROS2 Jazzy et RViz	17
6.2 Tentative de scan matching par ICP	18
7. Problèmes rencontrés, essais et solutions	19
7.1 Bruit électromagnétique et échec de la tachymétrie de phase	19
7.2 Dérive gyroscopique	19
7.3 Latence de boucle et complexité de ROS2	19
7.4 Coût du réglage des PID par essais-erreurs	19
7.5 Échec du scan matching et repli sur le FTG pur	20
7.6 Synthèse des problèmes et résolutions.....	20
8. Réalisation et intégration	21
8.1 Plaque support conçue en CAO et imprimée en 3D	21
8.2 Câblage et carte de prototypage.....	21
8.3 Sécurité des batteries.....	22
9. Performances et résultats	22
9.1 Domaine de fonctionnement validé	22
9.2 Indicateurs clés.....	22
10. Conclusion et perspectives.....	23
10.1 Ce qui nous démarque.....	23
10.2 Limites identifiées.....	23
10.3 Perspectives d'amélioration.....	23
10.4 Bilan personnel et apprentissages d'équipe	24
Glossaire	24

1. Introduction et contexte

1.1 Le défi AWAKE Challenge 2026

L'AWAKE Challenge est un concours de créativité technologique organisé par AWAKE Group (Innovateam) qui oppose les écoles d'ingénieurs françaises. Pour sa cinquième édition, en 2026, le défi consiste à concevoir, dimensionner et construire un mini-véhicule autonome à l'échelle 1/10ème, capable de parcourir en totale autonomie un circuit révélé le jour de la compétition. La course se déroule en intérieur, dans un gymnase, sur une piste de 80 cm de largeur délimitée par des parois de tuyau de VMC de 16 cm de hauteur, sur une surface de 15×8 m au sol foncé.

La performance est évaluée sur cinq tours consécutifs en contre-la-montre, le temps final étant la somme des cinq tours. La notation combine cinq composantes de 20 points chacune : distance parcourue, meilleur temps, choix technologiques, qualité du projet et originalité. Chaque contact avec une bordure entraînant une déformation, ou toute intervention manuelle, est pénalisé de deux points. La régularité et la fiabilité priment donc autant que la vitesse pure.

Le règlement impose une motorisation électrique, l'intégration d'un LiDAR comme technologie de référence et d'un minimum de trois capteurs à ultrasons. Le GPS est interdit (course en intérieur). Les centrales inertiels et accéléromètres sont autorisés et valorisés. Le budget alloué est de 900 €, remboursé sur présentation des factures.

1.2 Objectifs du projet

L'objectif central de l'équipe ScieWay était de produire un véhicule capable de réaliser les cinq tours réglementaires sans aucune intervention ni contact avec les bordures, en privilégiant la fiabilité absolue avant la vitesse de pointe. Cette priorité découle directement de la grille de notation : un véhicule rapide mais qui sort de piste perd davantage qu'un véhicule régulier et lent.

Nous avons décliné cet objectif principal en plusieurs objectifs techniques :

- concevoir une architecture matérielle et logicielle robuste, séparant clairement la perception haut niveau du contrôle temps réel ;
- intégrer et faire coopérer les capteurs imposés (LiDAR, ultrasons) et complémentaires (IMU, capteur à effet Hall) ;
- identifier expérimentalement le comportement de la chaîne de propulsion afin d'asservir précisément la vitesse et la direction ;
- implémenter un algorithme de navigation autonome capable de gérer un circuit inconnu, sans carte préalable ;

1.3 Périmètre et démarche

Le périmètre du projet couvre l'intégralité du cycle d'ingénierie : analyse du besoin, choix des composants, conception mécanique de l'intégration, développement électronique, identification et modélisation, synthèse de la commande, programmation embarquée, et validation expérimentale. La conception du châssis lui-même est hors périmètre : le règlement autorise l'achat d'un châssis

commercial, ce que nous avons fait pour concentrer l'effort sur l'intelligence embarquée, conformément à l'esprit du concours qui exige que l'ordinateur embarqué, la gestion d'alimentation et les capteurs soient développés et intégrés par les étudiants.

La démarche suivie a été résolument itérative et expérimentale : plutôt que de figer une solution théorique, nous avons procédé par essais, mesures et corrections successives. Cette approche nous a conduits à abandonner certaines pistes (tachymétrie de phase, scan matching) au profit de solutions plus robustes, ce qui constitue l'un des apprentissages majeurs du projet et fait l'objet d'une section dédiée.

1.4 Présentation de l'équipe et répartition des rôles

L'équipe ScieWay est composée de trois étudiants de deuxième année d'une même classe à Arts et Métiers. Pour mener un projet pluridisciplinaire sur un calendrier contraint, nous avons réparti les responsabilités par domaine d'expertise tout en maintenant une vision système partagée par des revues techniques régulières et un dépôt GitHub centralisé.

Tableau 1 : Répartition des rôles au sein de l'équipe ScieWay.

Membre	Rôle principal	Responsabilités
AZZAL Haitam	Chef de projet + Algorithme	Management, finance, achat des composants, planification, communication, co-responsable algorithme
BAHIR El Mahdi	Dév. électronique & logiciel	Architecture matérielle, intégration capteurs, boucles d'asservissement vitesse & direction, responsable algorithme
AKBIB BUKRABA Youssef	Conception & réalisation + Algo	Analyse du besoin, CAO, impression 3D, assemblage et modélisation système, co-responsable algorithme

Cette organisation a permis une progression parallèle sur les trois axes du projet — mécanique, électronique/logiciel et algorithmique — tout en évitant le cloisonnement grâce au caractère partagé du travail algorithmique.

2. Architecture du véhicule

2.1 Vue d'ensemble du système

Le véhicule repose sur une architecture distribuée à deux niveaux, dimensionnée pour placer chaque tâche sur le processeur le plus adapté. Le Raspberry Pi 5 assure la perception et la décision haut niveau : acquisition et traitement du flux LiDAR, exécution de l'algorithme Follow The Gap, fusion de capteurs et cartographie. Le microcontrôleur ESP32 prend en charge le contrôle bas niveau temps réel : génération des signaux PWM, asservissement de la vitesse et de la direction, lecture des capteurs critiques et gestion de la sécurité.

Les deux calculateurs communiquent par liaison série UART, le Raspberry Pi transmettant à l'ESP32 un couple de consignes (angle du gap détecté, vitesse cible) à chaque scan exploité. Cette séparation présente un double avantage : elle décharge le Raspberry Pi des contraintes dures de temps réel, peu compatibles avec un système d'exploitation Linux non temps réel, et elle confie l'actuation à un microcontrôleur déterministe capable de générer un PWM stable et de réagir à une interruption en moins d'une milliseconde.

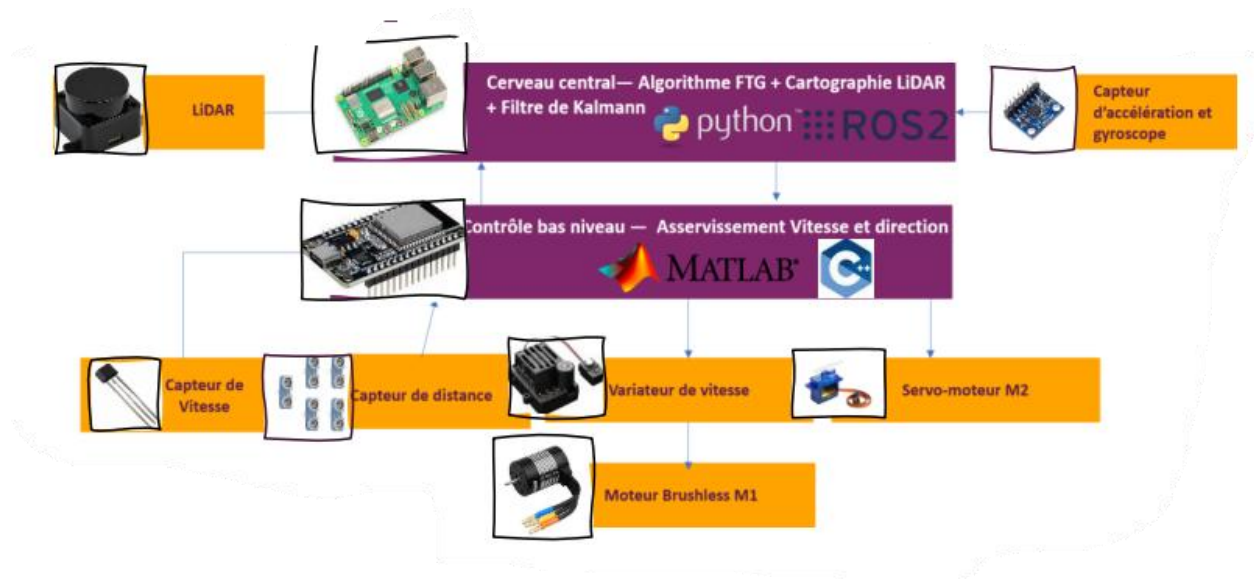


Figure 2 : Architecture système distribuée : haut niveau (Raspberry Pi 5) et bas niveau (ESP32).

Le LiDAR LD19 et la centrale inertielle MPU-6050 alimentent le Raspberry Pi. L'ESP32 lit le capteur à effet Hall (odométrie) et les quatre capteurs ultrasons, et pilote l'ESC (donc le moteur brushless) ainsi que le servomoteur de direction.

2.2 Caractéristiques mécaniques et électriques

La base mécanique est un châssis commercial RC 1/10ème quatre roues motrices, retenu pour sa robustesse et sa transmission métallique. L'option 4WD est recommandée par le règlement pour la motricité sur le revêtement du gymnase. L'ensemble équipé pèse 1,6 kg.

Tableau 2 : Caractéristiques mécaniques et électriques du véhicule.

Caractéristique	Valeur retenue
Échelle / transmission	1/10ème, 4 roues motrices (4WD)
Dimensions (L × l × h)	320 × 153 × 120 mm (sous les maxima 450 × 200 × 200 mm)
Masse à vide équipée	1,6 kg
Châssis	Alliage d'aluminium 7075, transmission métallique
Moteur	Brushless 3700 KV (sans capteur)
Variateur (ESC)	E45P — 45 A, BEC intégré 5 V / 3 A
Batterie	LiPo 2S — 7,4 V / 2000 mAh
Vitesse théorique max (matériel)	≈ 10 m/s
Vitesse de croisière validée	1,5 m/s en autonomie complète
Autonomie	10 à 13 min de course continue

Le dimensionnement laisse une marge confortable sur les dimensions réglementaires, ce qui facilite l'intégration de l'électronique tout en gardant un centre de gravité bas. Le rapport entre vitesse matérielle théorique (≈ 10 m/s) et vitesse exploitée (1,5 m/s) traduit la limite logicielle de localisation discutée plus loin : le matériel n'est pas le facteur limitant.

2.3 Chaîne de perception et de calcul

Le tableau suivant récapitule les modules de perception et de calcul, leur interface et leur fonction.

Tableau 3 : Modules de perception et de calcul embarqués.

Module	Interface	Fonction & caractéristiques clés
LiDAR LD19	UART	Vision 360°, portée 12 m, 10 Hz, ±3 cm, résistant à 30 000 lux
4 × HC-SR04	GPIO	Anti-collision proche, 2–400 cm, sécurité zone aveugle
MPU-6050	I2C	IMU 6 axes (accéléromètre + gyroscope), fusion Kalman
Capteur Hall A3144	GPIO (IRQ)	2 aimants sur l'arbre de transmission, odométrie
Raspberry Pi 5 (8 Go)	USB / GPIO	FTG, traitement LiDAR, cartographie ROS2
ESP32	UART / PWM	Contrôle PWM, PID vitesse/direction, sécurité

2.4 Pile logicielle

La pile logicielle combine plusieurs environnements, chacun choisi pour sa pertinence sur une couche donnée :

- Python 3.11 sur le Raspberry Pi 5 — pipeline perception → décision (lecture LiDAR, Follow The Gap, fusion) ;
- C++ / Arduino sur l'ESP32 — contrôle bas niveau temps réel (PWM, PID, interruptions) ;
- ROS2 Jazzy (Ubuntu 24.04 LTS) — visualisation RViz du flux LiDAR et débogage de la cartographie ;
- MATLAB — réglage (tuning) des correcteurs PID sur le modèle identifié ;
- protocole UART au format texte « angle,rpm » entre Raspberry Pi et ESP32.

Un choix d'architecture important mérite d'être souligné : ROS2 Jazzy est volontairement cantonné à la visualisation et au débogage de la cartographie. Le pipeline de commande proprement dit (perception → décision → actuation) reste hors ROS, en Python natif. Ce choix réduit la latence de boucle et la complexité globale ; il a été motivé par le fait que nous avons découvert ROS2 à l'occasion du scan matching, et qu'imposer le passage par l'intergiciel DDS à toute la commande aurait introduit une latence de l'ordre de 15 à 20 ms, contre moins de 2 ms en Python natif.

3. Capteurs et perception

La perception est le cœur d'un véhicule autonome : la qualité des décisions de navigation dépend directement de la fiabilité des mesures. ScieWay combine quatre familles de capteurs aux rôles complémentaires : le LiDAR pour la perception de l'environnement à 360°, les ultrasons pour la sécurité de proximité, l'IMU pour l'estimation d'orientation et le capteur à effet Hall pour l'odométrie.

3.1 LiDAR LD19 — perception principale

Le LiDAR LD19 est la technologie de référence imposée par le règlement. Il fournit un balayage à 360° à une fréquence de rotation d'environ 10 Hz, avec une portée annoncée de 12 m et une précision de l'ordre de ± 3 cm. Sa résistance à 30 000 lux est un argument important pour une utilisation en gymnase, où les baies vitrées peuvent provoquer de forts contrastes de luminosité.

Le LD19 communique en UART à 230 400 bauds selon un protocole propriétaire par paquets. Chaque paquet de 47 octets contient un en-tête, la vitesse de rotation, un angle de début et de fin, douze mesures (distance sur 16 bits et indice de confiance sur 8 bits), un horodatage et un octet de contrôle CRC. Nous avons implémenté en Python le décodage complet de ce protocole, incluant la vérification d'intégrité par table CRC précalculée, la synchronisation sur l'octet d'en-tête et l'interpolation linéaire des angles entre le début et la fin de chaque paquet.

```
def parse_packet(raw):
    if raw[0] != HEADER or raw[1] != VERLEN:
        return None
    if calc_crc(raw[:PKT_SIZE-1]) != raw[PKT_SIZE-1]:
        return None # rejet du paquet corrompu
    start = u16(raw,4)*0.01 # angle début (deg)
    end = u16(raw,42)*0.01 # angle fin (deg)
    if end < start: end += 360.0 # passage par 0°/360°
    step = (end-start)/(NB_POINTS-1)
    pts = []
    for i in range(NB_POINTS):
        d = u16(raw, 6+i*3)
        a = math.radians((start+i*step) % 360.0)
        if 0 < d < 2000: # filtrage des aberrations
            pts.append((a, d))
    return pts
```

La vérification CRC est essentielle : sur un lien série à haut débit, les paquets corrompus sont fréquents et un point erroné peut faire détecter un faux obstacle ou un faux passage libre. Le filtrage

des distances aberrantes (mesures nulles ou supérieures à 2 m) élimine les artefacts les plus courants en amont de l'algorithme de navigation.

3.2 Capteurs ultrasons HC-SR04 — sécurité de proximité

Quatre capteurs ultrasons HC-SR04 sont disposés autour du véhicule (gauche, droite, avant-gauche, avant-droit). Leur rôle n'est pas la navigation mais la sécurité : ils couvrent la zone aveugle du LiDAR, à très courte distance et notamment frontalement, où la résolution angulaire du LD19 et sa fréquence de rafraîchissement à 10 Hz ne suffisent pas à garantir l'absence de collision à pleine vitesse.

Les quatre capteurs sont lus directement par l'ESP32 sur des paires de broches GPIO dédiées. Ce câblage sur le microcontrôleur bas niveau, et non sur le Raspberry Pi, est délibéré : il rend la réaction de sécurité indépendante du pipeline Python et donc beaucoup plus rapide et plus déterministe.

```
Ultrasonic Ug(12, 13); // gauche
Ultrasonic Ud(14, 27); // droite
Ultrasonic Uag(26, 25); // avant-gauche
Ultrasonic Uad(33, 32); // avant-droit
```

3.3 Centrale inertielle MPU-6050

La centrale inertielle MPU-6050 fournit l'accélération sur trois axes et la vitesse angulaire (gyroscope) sur trois axes, par liaison I2C. Elle alimente le filtre de Kalman décrit au chapitre 4. Son apport principal est l'estimation de la vitesse de lacet (rotation autour de l'axe vertical), utile pour estimer l'orientation du véhicule entre deux mises à jour de position. Sa limite bien connue est la dérive du gyroscope : intégrée dans le temps, la vitesse angulaire produit une erreur d'orientation croissante, mesurée à environ 4° en deux minutes, qu'il faut corriger par fusion.

3.4 Capteur à effet Hall — odométrie de précision

La mesure de la vitesse réelle des roues est indispensable pour fermer la boucle d'asservissement de vitesse. Notre première approche, la tachymétrie de phase (lecture des forces contre-électromotrices induites sur les phases du moteur brushless), s'est révélée inexploitable : le bruit électromagnétique généré par la commutation haute fréquence de l'ESC rend les mesures instables. Cet échec, détaillé au chapitre 7, nous a conduits à une solution mécanique robuste.

Nous avons fixé deux aimants diamétralement opposés sur l'arbre de transmission, en regard d'un capteur à effet Hall A3144 monté sur le châssis. Chaque tour génère ainsi deux impulsions, comptées sur interruption matérielle de l'ESP32 (broche GPIO 34), ce qui garantit l'absence de perte d'impulsion même à haute vitesse de rotation.

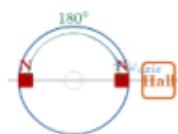


Figure 3 : Principe du capteur à effet Hall : deux aimants à 180° génèrent deux impulsions TTL par tour.

```
volatile uint32_t hallCount = 0;
volatile uint32_t lastHallUs = 0;
void IRAM_ATTR hallISR() { // ISR placée en RAM, rapide
    hallCount++;
    lastHallUs = micros();
}
// attachée sur fronts montants ET descendants :
attachInterrupt(PIN_HALL, hallISR, FALLING);
attachInterrupt(PIN_HALL, hallISR, RISING);
```

Le traitement par interruption (ISR placée en mémoire interne via `IRAM_ATTR`) assure une latence inférieure à la milliseconde et l'absence de perte de comptage. La vitesse de rotation est ensuite calculée périodiquement (au plus 20 fois par seconde) à partir du nombre d'impulsions accumulées et du temps écoulé, ce qui lisse le bruit de comptage tout en restant suffisamment réactif pour l'asservissement.

Au-delà de la mesure de vitesse, cette odométrie apporte deux fonctions précieuses : la détection de patinage (commande PWM élevée mais vitesse mesurée faible) et le comptage des tours de roue, exploitable pour la stratégie de course. L'erreur de vitesse résultante est estimée à $\pm 2\%$.

4. Identification et modélisation

Pour asservir précisément la vitesse et la direction, il fallait disposer d'un modèle du véhicule. Plutôt que de partir de valeurs constructeur incertaines, nous avons identifié expérimentalement le comportement réel du système, puis construit un modèle cinématique et dynamique exploité pour le réglage de la commande et la fusion de capteurs.

4.1 Identification expérimentale de la chaîne de propulsion

La première campagne d'essais a caractérisé la relation entre la consigne PWM appliquée à l'ESC et la vitesse linéaire du véhicule. Un capteur ultrason placé en regard d'un mur de référence a mesuré la position du véhicule au cours du temps pour différentes consignes PWM. La dérivée de la position fournit la vitesse, et le recoupement avec la vitesse de rotation mesurée par le capteur Hall donne le rapport de transmission global.

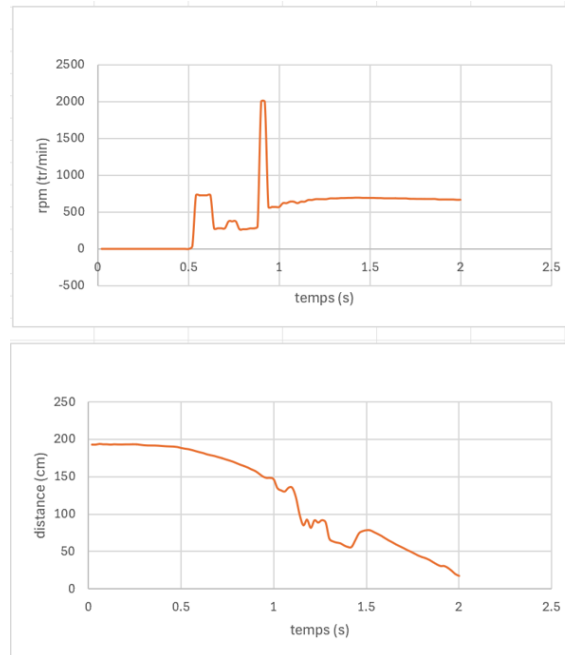


Figure 4 : Réponses expérimentales : vitesse de rotation (haut) et position mesurée par ultrason (bas) au cours du temps.



Figure 5 : Dispositif d'identification : mesure de déplacement face à un mur de référence.

Cette campagne a fourni une caractéristique statique PWM $\rightarrow$ vitesse linéaire ainsi qu'un rapport de transmission global $r \approx 0,0179$ (en rpm par m/s). Un modèle linéaire simplifié de la chaîne de propulsion en a été déduit, suffisant pour le réglage du correcteur de vitesse autour du point de fonctionnement nominal (1 à 1,5 m/s).

4.2 Modélisation cinématique : modèle bicyclette d'Ackermann

La direction du véhicule, de type Ackermann (les roues avant braquent, les roues arrière suivent), est modélisée par un modèle bicyclette : les deux roues d'un même essieu sont ramenées à une roue équivalente sur l'axe longitudinal. Ce modèle relie la vitesse de lacet $\dot{\psi}$ à la vitesse longitudinale V et à l'angle de braquage δ_F par la loi d'Ackermann.

$$\dot{\psi} = \left(\frac{V}{L}\right) \cdot \tan(\delta_F) \quad \text{d'où} \quad R = \frac{L}{\tan(\delta_F)}$$

où L est l'empattement et R le rayon de courbure instantané. Cette relation est fondamentale pour la commande : connaissant le rayon que l'on souhaite suivre, on en déduit directement l'angle de braquage à appliquer.

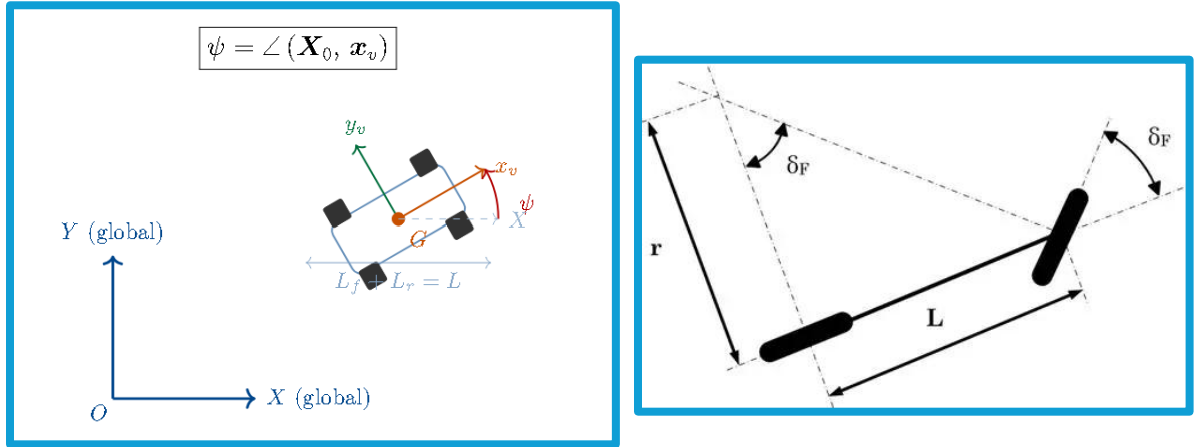


Figure 6 : Modèle cinématique : repère global/véhicule (gauche) et géométrie d'Ackermann (droite).

La caractérisation géométrique du véhicule donne un angle de braquage maximal $\delta_{\max} \approx 30^\circ$ et un rayon de braquage minimal $R_{\min} \approx 30$ cm. Ces valeurs sont compatibles avec une piste de 80 cm de largeur : le véhicule peut négocier les virages serrés du circuit sans avoir à manœuvrer.

4.3 Modèle dynamique et calcul de l'angle de commande

Le modèle cinématique a été enrichi d'un modèle dynamique intégrant l'inertie, les frottements et le glissement pneumatique en virage, afin de prédire l'accélération maximale admissible et la vitesse critique d'entrée en virage. Ce modèle permet d'éviter de demander au véhicule une trajectoire physiquement irréalisable, source de sous-virage et de sortie de piste.

À partir du couple (distance d , angle θ) du passage libre détecté par le LiDAR, nous calculons l'angle de braquage à appliquer. En supposant que le véhicule doit rejoindre le point cible par un arc de cercle, la géométrie conduit à un rayon $R = d / (2 \cdot \sin \theta)$, puis à l'angle de braquage :

$$\delta_F = \arctan\left(2 \cdot L \cdot \frac{\sin(\theta)}{d}\right)$$

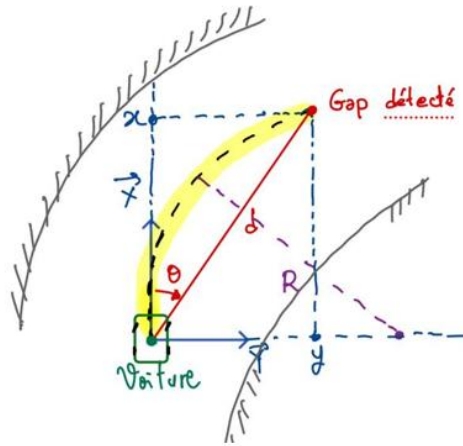


Figure 7 : Géométrie du calcul de l'angle de commande à partir du gap détecté par le LiDAR.

Cette formule établit le lien direct entre la perception (gap LiDAR) et l'actuation (angle servo) : à chaque scan, le couple (distance, angle) du meilleur passage est converti en consigne d'angle de braquage transmise à l'ESP32.

4.4 Fusion de capteurs par filtre de Kalman

Aucun capteur pris isolément ne fournit une estimation fiable de l'état complet du véhicule : le gyroscope dérive, l'odométrie Hall ignore le glissement, et le modèle dynamique accumule les erreurs de modélisation. La fusion par filtre de Kalman combine ces sources pour produire une estimation optimale de l'état (position, vitesse linéaire, vitesse angulaire, biais gyroscopique).

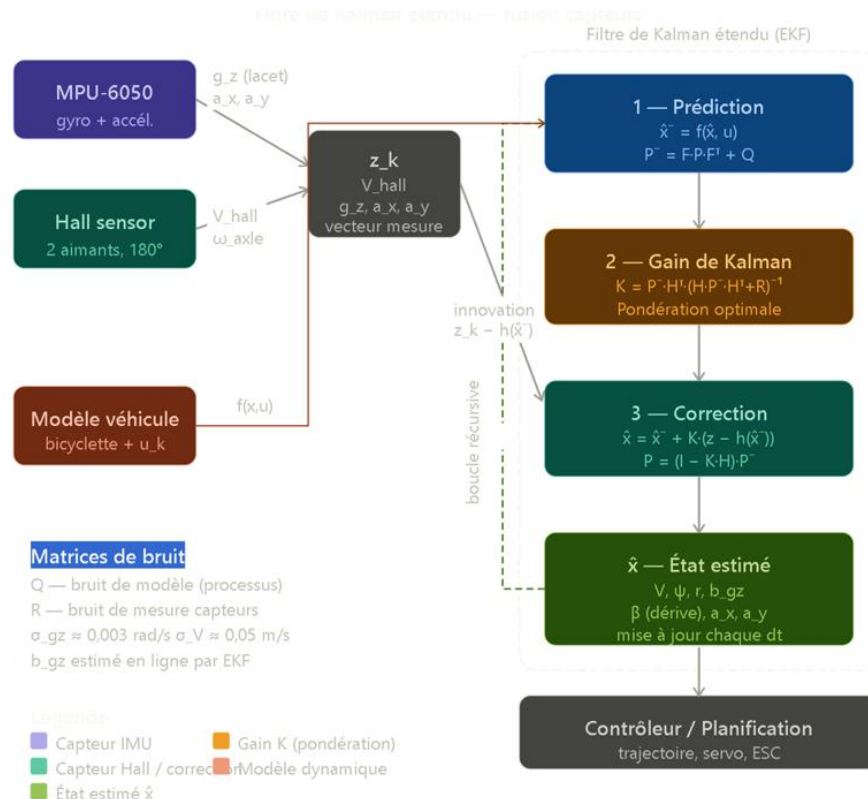


Figure 8 : Structure du filtre de Kalman étendu : prédiction (modèle), gain de Kalman, correction (mesures).

Le filtre fonctionne en deux phases récursives. La phase de prédiction projette l'état à l'aide du modèle bicyclette et de la commande. La phase de correction confronte cette prédiction au vecteur de mesure (vitesse Hall, vitesse de lacet et accélérations de l'IMU) et corrige l'estimation par le gain de Kalman, qui pondère optimalement modèle et mesures selon leurs covariances de bruit respectives. Le biais du gyroscope est estimé en ligne, ce qui réduit la dérive d'orientation de 4° en deux minutes à moins de 1°.

Tableau 4 : Paramètres de bruit retenus pour le filtre de Kalman.

Grandeur	Valeur
Bruit de mesure vitesse de lacet σ_{gz}	$\approx 0,003$ rad/s
Bruit de mesure vitesse linéaire σ_V	$\approx 0,05$ m/s
Biais gyroscopique b_{gz}	estimé en ligne par le filtre
Dérive d'orientation après fusion	$< 1^\circ$ (contre 4° en 2 min sans fusion)

5. Synthèse de la commande : Follow The Gap et PID

La commande globale articule deux niveaux. Au niveau haut, l'algorithme Follow The Gap (FTG) traite le flux LiDAR pour décider, à chaque scan, du cap et de la vitesse cible. Au niveau bas, deux correcteurs PID indépendants asservissent la vitesse et la direction sur les consignes issues du FTG. C'est cette commande réactive FTG, sans carte préalable, qui constitue le mode de navigation finalement retenu et validé.

5.1 Pourquoi Follow The Gap ?

Le circuit étant révélé le jour de la compétition, une stratégie reposant sur une carte construite à l'avance était exclue. Le Follow The Gap est une méthode de navigation réactive : il ne nécessite aucune connaissance préalable de l'environnement et décide uniquement à partir du scan courant. C'est une référence éprouvée dans les compétitions de voitures autonomes miniatures de type F1TENTH, ce qui en faisait un choix à la fois robuste et adapté à notre contrainte de circuit inconnu.

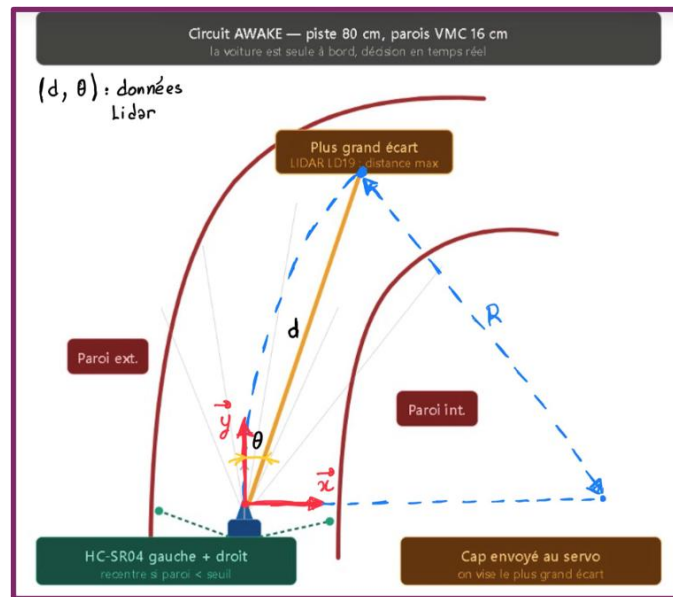


Figure 9 : Principe FTG sur le circuit AWAKE : le véhicule vise le plus grand espace libre détecté.

5.2 L'algorithme Follow The Gap pas à pas

L'algorithme se décompose en six étapes appliquées à chaque scan exploité :

1. Troncature du scan — seuls les points situés dans le secteur frontal sont conservés ; l'arrière du véhicule est ignoré car non pertinent pour la progression.
2. Pré-traitement — les valeurs invalides sont remises à zéro, les distances infinies plafonnées, et un filtre de moyenne glissante réduit le bruit du LD19.
3. Point le plus proche — l'obstacle le plus dangereux est identifié par recherche du minimum de distance (argmin).

4. Bulle de sécurité (safety bubble) — un rayon de sécurité autour de ce point le plus proche est mis à zéro, créant une zone interdite qui protège la largeur du véhicule.
5. Recherche du gap — la plus longue séquence de points non nuls est identifiée comme l'espace libre maximal ; le cap visé est le centre de ce gap.
6. Braquage et vitesse — l'angle de braquage est proportionnel à l'écart entre le cap et l'axe du véhicule ; la vitesse est modulée selon l'angle (réduite en virage serré, augmentée en ligne droite).

Dans notre implémentation finale, la sélection du gap est affinée par un raisonnement sur la dispersion des mesures et la distance moyenne dans des secteurs angulaires : l'espace $[-90^\circ, +90^\circ]$ est divisé en intervalles, et chaque intervalle est évalué par sa distance moyenne (favorisée) et son écart-type (pénalisé). Le secteur retenu est celui qui maximise la profondeur tout en minimisant la dispersion, ce qui évite de viser un faux passage causé par quelques points isolés.

```
# sélection du meilleur secteur (distance haute, dispersion faible)
gap_info.sort(key=lambda x: (x[2], -x[3]), reverse=True)
best_gap = gap_info[0]
gap_center_angle = (best_gap[0] + best_gap[1]) / 2
# envoi de la consigne (angle, vitesse) vers l'ESP32 en UART
esp.write(f"{0.75*gap_center_angle},100\n".encode())
```

Le coefficient 0,75 appliqué à l'angle du gap est un gain de direction réglé expérimentalement : il atténue la consigne brute pour éviter les sur-braquages et les oscillations, au prix d'une réactivité légèrement réduite. La valeur de vitesse (ici 100, exprimée en unité de consigne moteur) est volontairement modérée pour rester dans le domaine de fonctionnement stable validé.

5.3 Asservissement bas niveau : PID vitesse et direction

Sur l'ESP32, deux correcteurs PID indépendants traduisent les consignes FTG en commandes d'actionneurs.

5.3.1 PID de vitesse

Le PID de vitesse reçoit la consigne issue du FTG, mesure la vitesse réelle par le capteur Hall et pilote la largeur d'impulsion PWM envoyée à l'ESC. La sortie est bornée pour ne jamais dépasser une valeur de sécurité et pour interdire la marche arrière en course. Les gains retenus sont $K_p = 0,8$, $K_i = 0,2$ et $K_d = 0,03$, et la sortie est limitée à la plage $[1500 ; 1650]$ μs autour du neutre.

```
float Kp = 0.8, Ki = 0.2, Kd = 0.03;
const double ESC_NEUTRE = 1500.0, ESC_MAX = 1650.0;
pidRpm.SetOutputLimits(0.0, ESC_MAX - ESC_NEUTRE);
pidRpm.SetSampleTime(100);           // 10 Hz
// ... à chaque boucle :
pid_rpm_input = rpmMeasure;           // mesure Hall
if (pidRpm.Compute()) {
    double escCommand = ESC_NEUTRE + pid_rpm_output;
    if (escCommand > ESC_MAX) escCommand = ESC_MAX;
    monESC.writeMicroseconds((int)escCommand);
}
```

L'erreur statique résultante reste inférieure à 5 % à 1,5 m/s, ce qui assure une vitesse de croisière stable et reproductible.

5.3.2 PID de direction

Le PID de direction asservit l'angle du servomoteur sur la consigne d'angle issue du FTG, le servo étant centré sur 90° et commandé en valeur relative autour de ce neutre. Le temps de réponse mesuré est d'environ 80 ms pour un échelon de 5° à 25°, et l'angle est borné à $\pm 30^\circ$ afin de préserver le couple mécanique et de rester dans la plage géométrique du modèle d'Ackermann.

```
monServo.write(90.0 + pid_angle_input); // neutre + consigne
```

5.4 Réglage des PID sous MATLAB

Le réglage des gains a d'abord été conduit hors ligne sous MATLAB, à partir du modèle identifié au chapitre 4, plutôt que par essais-erreurs directs sur le véhicule. La démarche a été la suivante :

1. conversion du modèle identifié en fonction de transfert $H(s)$;
2. réglage avec l'outil PID Tuner dans le domaine fréquentiel ;
3. respect des critères de dépassement $< 10\%$ et de temps de réponse $< 200\text{ ms}$;
4. recherche d'un optimum sur le couple (vitesse, angle) ;
5. validation par simulation sur circuit test ;
6. ajustement fin expérimental sur le circuit réel.

Cette approche modèle a fait gagner un temps considérable par rapport au réglage purement empirique, qui se révélait coûteux et peu reproductible : chaque essai sur véhicule mobilise toute l'équipe, use la mécanique et la batterie, et fournit des mesures bruitées.

5.5 Sécurité multi-capteurs

La sécurité repose sur trois couches superposées, conçues pour être redondantes et indépendantes les unes des autres.

Couche 1 — LiDAR (principale)

Le FTG maintient en permanence le véhicule dans l'espace libre du scan. La détection couvre de 0,3 m à 12 m, rafraîchie à 10 Hz, et la bulle de sécurité autour de l'obstacle le plus proche garantit une marge latérale.

Couche 2 — Ultrasons (redondance)

Si un capteur HC-SR04 détecte un obstacle à moins de 10 cm dans la trajectoire, un signal de recentrage est appliqué indépendamment du pipeline Python : les ultrasons prennent le relais pour ramener le véhicule vers le milieu du circuit. Cette couche couvre la zone aveugle frontale du LiDAR (environ 15 cm) et prévient les collisions frontales non détectées par la perception principale.

Couche 3 — Limites logicielles

Enfin, des garde-fous logiciels bornent le système : vitesse maximale plafonnée dans le code FTG (10 Hz de LiDAR suffisent à cette vitesse), angle de servo limité à $\pm 30^\circ$, et chien de garde (watchdog) qui déclenche un arrêt d'urgence en l'absence de scan LiDAR depuis 500 ms.

6. Cartographie LiDAR et tentative de localisation

En parallèle de la commande réactive, nous avons exploré une cartographie temps réel du circuit, dans l'espoir de débloquent une navigation à plus haute vitesse par localisation absolue. Cette partie a constitué notre première expérience de ROS2 et a abouti à un résultat partiel, instructif, qui justifie le repli final sur le FTG pur.

6.1 Visualisation sous ROS2 Jazzy et RViz

Le flux LiDAR est publié sous ROS2 Jazzy (sur le Raspberry Pi 5 sous Ubuntu 24.04 LTS) et visualisé sous RViz. RViz superpose le scan LiDAR à la trajectoire reconstituée, chaque point étant repéré en temps réel dans le repère monde. La précision de la cartographie locale atteint ± 3 cm sur 5 m, avec une reconstruction fidèle de la géométrie du circuit et une superposition scan-à-scan correcte à l'œil nu.

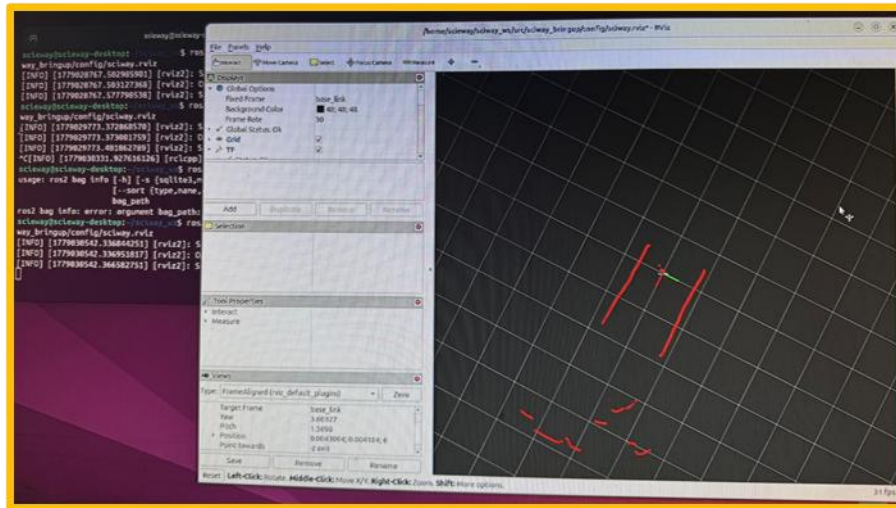


Figure 10 : Visualisation du scan LiDAR et de la trajectoire reconstituée sous RViz (ROS2 Jazzy).

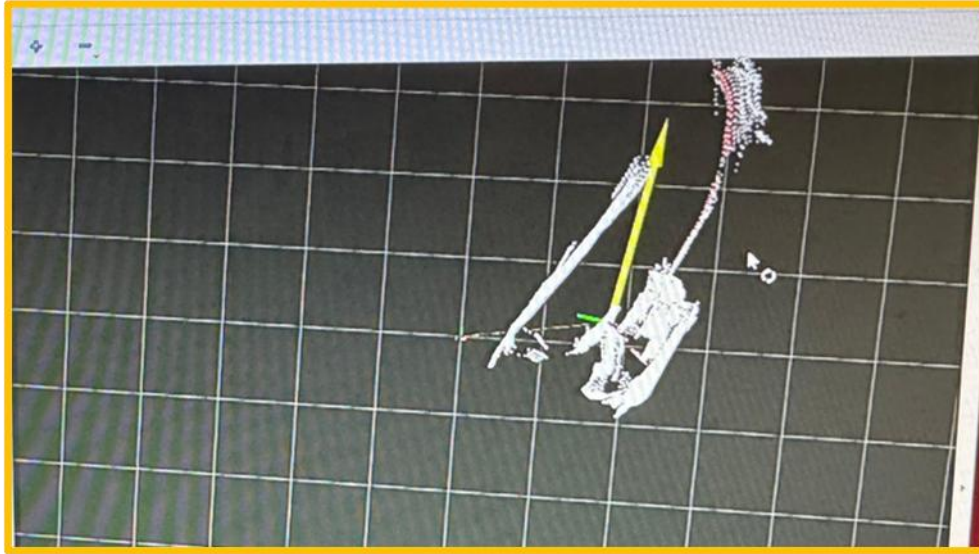


Figure 11 : Résultats de cartographie : nuage de points LiDAR

Il faut souligner que ROS2 est ici cantonné à la cartographie et au débogage : le pipeline de commande reste hors ROS pour préserver une latence faible (< 2 ms contre 15 à 20 ms avec l'intergiciel DDS).

6.2 Tentative de scan matching par ICP

Pour fermer la boucle de localisation, nous avons tenté un algorithme de scan matching de type ICP simplifié (Iterative Closest Point). Le principe consiste à associer les points d'un scan à ceux du scan suivant pour estimer le déplacement du véhicule entre les deux, puis à composer ces transformations successives pour reconstruire la pose absolue.

Les résultats n'ont pas atteint le niveau de fiabilité requis. Chaque appariement introduit une petite erreur de quelques centimètres ; comme les transformations se composent, ces erreurs s'accumulent. Sur cinq tours de circuit, le cumul produit une dérive métrique qui déforme la trajectoire estimée et rend la localisation inexploitable pour une navigation longue durée. Cette limite, fondamentale pour l'odométrie LiDAR sans fermeture de boucle, est analysée en détail au chapitre suivant.

7. Problèmes rencontrés, essais et solutions

Cette section rassemble, de façon assumée, les difficultés rencontrées et les pistes explorées — y compris celles qui n’ont pas abouti. Le choix de documenter les échecs n’est pas anecdotique : il reflète la réalité d’un cycle d’ingénierie et constitue, de notre point de vue, l’apprentissage le plus formateur du projet. Savoir abandonner une voie qui ne fonctionne pas au profit d’une solution plus robuste est un réflexe d’ingénieur que cette compétition nous a aidés à intégrer.

7.1 Bruit électromagnétique et échec de la tachymétrie de phase

Pour mesurer la vitesse du moteur, notre première idée était la tachymétrie de phase : lire les forces contre-électromotrices induites sur les phases du moteur brushless pour en déduire la vitesse de rotation, sans capteur additionnel. Cette solution élégante s’est heurtée à une réalité physique : la commutation haute fréquence de l’ESC génère un bruit électromagnétique qui rend le signal de phase inexploitable. Les mesures étaient instables et impossibles à filtrer de manière fiable.

Solution retenue : nous avons basculé vers une mesure mécanique robuste par capteur à effet Hall et deux aimants sur l’arbre de transmission (chapitre 3.4). Le compromis — l’ajout d’une pièce mécanique et de deux aimants — est largement compensé par la fiabilité obtenue (erreur de vitesse $\pm 2\%$, latence < 1 ms).

7.2 Dérive gyroscopique

La centrale inertielle MPU-6050, utilisée seule, présente une dérive d’orientation d’environ 4° en deux minutes, due à l’intégration du bruit et du biais du gyroscope. À l’échelle d’une course de cinq tours, cette dérive fausserait toute estimation d’orientation.

Solution retenue : la fusion par filtre de Kalman des mesures IMU et de l’odométrie Hall, avec estimation en ligne du biais gyroscopique, ramène la dérive sous 1° (chapitre 4.4).

7.3 Latence de boucle et complexité de ROS2

Notre intention initiale était de bâtir tout le pipeline (perception, décision, actuation) sous ROS2, par souci d’élégance architecturale. Nous avons rapidement constaté que l’intergiciel DDS de ROS2 introduit une latence de l’ordre de 15 à 20 ms, incompatible avec un asservissement réactif à haute fréquence, et que la complexité de mise en œuvre était disproportionnée pour notre besoin et notre niveau d’expérience initial sur ROS2.

Solution retenue : une architecture distribuée où le pipeline de commande reste en Python natif sur le Raspberry Pi (latence < 2 ms) et où ROS2 est cantonné à la visualisation et à la cartographie sous RViz. Chaque tâche est ainsi placée sur l’outil le plus adapté.

7.4 Coût du réglage des PID par essais-erreurs

Le réglage des correcteurs directement sur le véhicule est coûteux : chaque essai mobilise l’équipe, sollicite la mécanique et la batterie, et produit des mesures bruitées difficiles à comparer d’un essai à l’autre.

Solution retenue : le réglage hors ligne sous MATLAB sur le modèle identifié, validé en simulation puis affiné finement sur le circuit réel (chapitre 5.4).

7.5 Échec du scan matching et repli sur le FTG pur

La tentative de localisation par scan matching ICP est notre limite principale assumée. L'accumulation d'erreurs à chaque itération produit une dérive incompatible avec une navigation longue durée (chapitre 6.2). En l'absence de localisation absolue fiable, il devient impossible d'optimiser une trajectoire de référence (line racing) et de pousser la vitesse en toute sécurité.

Conséquence directe : le véhicule fonctionne uniquement en mode réactif (FTG pur), ce qui plafonne la vitesse exploitable de façon sûre à environ 1,5 à 2 m/s, alors que le matériel (moteur 3700 KV, châssis 7075) supporte jusqu'à 10 m/s — un facteur 5 de marge inexploitée. Ce repli est néanmoins un choix lucide : un FTG pur fiable à 1,5 m/s vaut mieux, au regard de la grille de notation, qu'une navigation rapide mais instable qui sortirait de piste.

7.6 Synthèse des problèmes et résolutions

Tableau 5 : Synthèse des problèmes rencontrés et des solutions apportées.

Problème	Cause	Solution / statut
Tachymétrie de phase inexploitable	Bruit EM de la commutation ESC	Capteur Hall + 2 aimants — résolu
Dérive gyroscopique ($4^\circ/2$ min)	Intégration biais gyroscope	Fusion Kalman IMU+Hall — résolu ($< 1^\circ$)
Latence de boucle (15–20 ms)	Intergiciel DDS de ROS2	Python natif + ROS2 limité à RViz — résolu (< 2 ms)
Tuning PID coûteux	Essais-erreurs sur véhicule	Tuning MATLAB sur modèle — résolu
Localisation par scan matching	Erreur ICP cumulée → dérive	Non finalisé — repli sur FTG pur

8. Réalisation et intégration

L'intégration matérielle a été un volet à part entière du projet. Au-delà du choix des composants, il fallait les agencer sur le châssis 1/10ème de façon compacte, stable et accessible, tout en respectant les dimensions réglementaires et en protégeant l'électronique des vibrations et des chocs.

8.1 Plaque support conçue en CAO et imprimée en 3D

Pour fixer proprement le Raspberry Pi, l'ESP32, la carte de prototypage et le LiDAR, nous avons conçu une plaque support sur mesure en CAO, puis imprimée en 3D. Elle offre des emplacements dédiés, un passage de câbles ordonné et un dégagement central, tout en surélevant le LiDAR pour garantir un champ de vision dégagé à 360°.

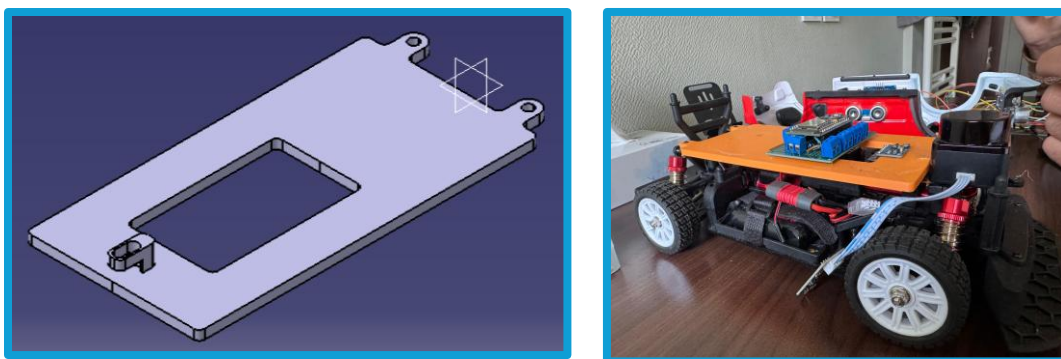


Figure 12 : Plaque support : modèle CAO (gauche) et intégration sur le châssis (droite).

8.2 Câblage et carte de prototypage

L'électronique a été câblée sur une carte de prototypage à double face, regroupant l'ESP32, les borniers de connexion des capteurs et la distribution d'alimentation. Cette centralisation facilite le diagnostic et permet de préparer une carte de rechange, conformément à la recommandation du règlement de prévoir une carte électronique de secours le jour de la compétition.



Figure 13 : Câblage sur une carte de prototypage.

La progression de l'intégration, du châssis nu au prototype carrossé, illustre la montée en maturité du véhicule au fil des points de suivi technique.

8.3 Sécurité des batteries

Conformément au règlement, les batteries LiPo sont manipulées et stockées dans un sac ignifugé prévu à cet effet pendant les pauses, et la carte est installée de manière à éviter tout contact avec la batterie ou les capteurs, pour prévenir les risques de surchauffe et de court-circuit.

9. Performances et résultats

Les essais ont été conduits sur un circuit d'essai exemplaire, reproduisant la configuration attendue du circuit de compétition AWAKE (piste de 80 cm délimitée par des tuyaux de VMC). Les résultats présentés sont reproductibles dans ces conditions ; le circuit de compétition étant révélé le jour J, les performances peuvent légèrement varier selon la configuration finale.

9.1 Domaine de fonctionnement validé

Trois régimes de fonctionnement ont été caractérisés selon la vitesse de consigne. Le régime nominal (0,5 à 1,5 m/s) est entièrement stable : le véhicule effectue les cinq tours réglementaires sans aucune intervention, sans contact avec les bordures et avec une trajectoire reproductible. Au-delà de 2 m/s, l'absence de boucle de localisation finalisée devient limitante ; au-delà de 3 m/s, le comportement devient instable.

Tableau 6 : Plages de vitesse et stabilité observée sur circuit d'essai.

Plage de vitesse	État	Commentaire
0,5 – 1,5 m/s	Stable	5 tours sans intervention, trajectoire reproductible
2 – 3 m/s	Limite	Marge de sécurité réduite, dépassements ponctuels
> 3 m/s	Instable	Réactivité FTG insuffisante sans localisation

9.2 Indicateurs clés

Le tableau suivant synthétise les performances finales mesurées sur le circuit d'essai.

Tableau 7 : Indicateurs de performance finaux.

Indicateur	Valeur
Vitesse de croisière validée	1,5 m/s en autonomie complète
Tours réalisés sans intervention	5 / 5 en régime nominal
Contacts avec les bordures	0 en régime nominal
Temps de décision FTG	< 5 ms par scan
Erreur statique de vitesse	< 5 % à 1,5 m/s
Temps de réponse direction	≈ 80 ms (5° → 25°)
Précision cartographie locale	±3 cm sur 5 m
Autonomie	10 à 13 min de course continue

Au regard de l'objectif initial — réaliser les cinq tours sans intervention ni contact — le résultat est atteint en régime nominal. La marge de performance non exploitée (matériel capable de 10 m/s) constitue la principale perspective d'amélioration.

10. Conclusion et perspectives

10.1 Ce qui nous démarque

Au terme du projet, le véhicule ScieWay se distingue par une démarche pensée pour la performance et la rigueur :

- une architecture distribuée plaçant la perception haut niveau sur le Raspberry Pi 5 et le contrôle temps réel sur l'ESP32, chaque tâche sur le bon processeur ;
- une fusion multi-capteurs (LiDAR, Hall, IMU, ultrasons) garantissant redondance et robustesse en compétition ;
- un algorithme Follow The Gap éprouvé, adapté de la référence F1TENTH, décidant en moins de 5 ms par scan et permettant cinq tours à 1,5 m/s sans intervention ;
- une démarche d'ingénierie complète et assumée, identification expérimentale comprise, jusqu'à la documentation lucide de ses limites.

10.2 Limites identifiées

La limite principale est le scan matching non finalisé : l'ICP simplifié cumule une erreur de localisation incompatible avec une navigation longue durée. Le véhicule fonctionne donc en mode réactif FTG pur, ce qui plafonne la vitesse exploitable à environ 1,5 à 2 m/s alors que le matériel supporte jusqu'à 10 m/s. L'absence de localisation absolue interdit par ailleurs l'optimisation d'une trajectoire de référence. Enfin, le comportement du LiDAR sous la luminosité réelle du gymnase (baies vitrées, sol foncé) reste à valider in situ, même si le LD19 est annoncé résistant à 30 000 lux.

10.3 Perspectives d'amélioration

Plusieurs pistes concrètes permettraient de débloquer la marge de performance inexploitée :

- Migration du scan matching vers NDT (Normal Distributions Transform) ou un SLAM à graphe de poses (pose graph) pour borner l'erreur cumulée et fermer la boucle de localisation ;
- Apprentissage de trajectoire (raceline learning) : apprendre la trajectoire idéale sur les premiers tours puis la suivre de façon optimisée ;
- Disparity Extender en complément du FTG, pour mieux anticiper les sorties de virage, avec un gain estimé de 10 à 15 % sur le temps au tour ;
- Mise à niveau du LiDAR vers une fréquence de 20 Hz (par exemple RPlidar A3) pour repousser la limite haute du domaine de vitesse stable.

Ces évolutions s'inscrivent dans une logique claire : la vitesse n'est pas bridée par la mécanique mais par la localisation. Lever ce verrou est la condition pour exploiter le facteur 5 de marge matérielle.

10.4 Bilan personnel et apprentissages d'équipe

Ce projet nous a confrontés à un cycle complet d'ingénierie : cahier des charges, dimensionnement, identification expérimentale, modélisation, synthèse de commande, intégration matérielle, mise au point logicielle et validation. La répartition par domaine d'expertise (système/électronique, conception/réalisation, algorithmes, management) a permis une progression parallèle, tout en maintenant une vision système partagée grâce à des revues techniques régulières et à un dépôt GitHub centralisé.

L'apprentissage le plus formateur a été de basculer sur une solution alternative face à un échec technique, plutôt que de s'obstiner. Renoncer à la tachymétrie de phase, puis au scan matching ICP, pour adopter des solutions plus robustes (capteur Hall, FTG pur) nous a appris à hiérarchiser fiabilité et ambition technique selon les véritables critères de réussite. Sur le plan humain, la coordination d'un projet pluridisciplinaire à trois, sous contrainte de calendrier et de budget, a été une expérience de gestion de projet aussi riche que le contenu technique lui-même.

Glossaire

FTG (Follow The Gap) : algorithme de navigation réactive visant le plus grand espace libre détecté par le LiDAR.

LiDAR : capteur de télémétrie laser fournissant un balayage de distances, ici à 360°.

ICP (Iterative Closest Point) : algorithme d'appariement de nuages de points pour estimer un déplacement.

NDT (Normal Distributions Transform) : méthode de recalage de scans bornant l'erreur, alternative à l'ICP.

SLAM : localisation et cartographie simultanées.

PID : correcteur Proportionnel-Intégral-Dérivé.

PWM : modulation de largeur d'impulsion, signal de commande des actionneurs.

ESC : variateur électronique de vitesse pilotant le moteur brushless.

IMU : centrale inertielle (accéléromètre + gyroscope).

Ackermann : géométrie de direction où les roues avant braquent et les arrières suivent.

ROS2 / RViz : intergiciel robotique et son outil de visualisation 3D.

BEC : régulateur intégré à l'ESC fournissant une tension stabilisée.